{EPITECH}

RUSH 1

< THE SQUARES />



RUSH 1

Preliminaries



Language: C
All of your source files, except for useless files (binary, temp files, obj files,...), must be included in your delivery.



- Don't push your `main` function into your delivery directory, we will be adding our own. Your files will be compiled adding our `main.c` and our `my_putchar.c` files.
- You are only allowed to use the `my_putchar` function to complete the following tasks, but **don't push it** into your delivery directory, and don't copy it in *any* of your delivered files.
- If one of your files prevents you from compiling with `*.c`, the Autograder will not be able to correct your work and you will receive a 0.



Segfault, bus error, floating exception are all grounds for disqualification!



Allowed function: `write`.

{EPITECH}

Goal

The goal of this project is to display a square on the screen. In each assignment, the squares will look different (see below).
You have to write the `rush` function, which will be called by **our** main function, which will look like this:

```
void rush(int x, int y);

int main()
{
        rush(5, 5);
        return (0);
}
```

In case of an error, the function should return and display `Invalid size\n` on the **standard error output** (the newline feed should of course be interpreted).



All assignments must be done.

Team work

Here are a couple of important rules to follow:

- ✓ The **team leader** must register their group for the oral presentation on the My.
- ✓ You must complete your oral presentation on Sunday, at the right time, and **with all of your partners**.
- ✓ Every member of the group should be fully aware of the work you will be presenting. Each member will be questioned, and your group's grade will be based on the worst explanations.
- ✓ You have to do everything within your power to contact your partners. **Excuses regarding group problems will not be accepted**.
  If, after you have tried **everything**, your partner is still unreachable, contact the local staff **ASAP**.

Assignment 1

Delivery: rush-1-1/*

Here is the expected display when calling `rush(5, 3)`:

```
Terminal
~$> clang *.c; ./a.out
o---o
|   |
o---o
```

When calling `rush(5, 1)`:

```
Terminal
~$> clang *.c; ./a.out
o---o
```

When calling `rush(1, 1)`:

```
Terminal
~$> clang *.c; ./a.out
o
```

When calling `rush(1, 5)`:

```
Terminal
~$> clang *.c; ./a.out
o
|
|
|
o
```


{EPITECH}

When calling `rush(4, 4)`:

```
Terminal
~$> clang *.c; ./a.out
o--o
|  |
|  |
o--o
```



A test binary is available next to this subject on My. **Usage: ./rush1-1 x y**

Assignment 2

Delivery: rush-1-2/*

Here is the expected display when calling `rush(5, 3)`:

```
Terminal
~$> clang *.c; ./a.out
/***\
*   *
\***/
```

When calling `rush(5, 1)`:

```
Terminal
~$> clang *.c; ./a.out
*****
```

When calling `rush(1, 1)`:

```
Terminal
~$> clang *.c; ./a.out
*
```

When calling `rush(1, 5)`:

```
Terminal
~$> clang *.c; ./a.out
*
*
*
*
*
```


{EPITECH}

When calling `rush(4, 4)`:

```
Terminal
~$> clang *.c; ./a.out
/**\
*  *
*  *
\**/
```



A test binary is available next to this subject on My. **Usage: ./rush1-2 x y**

Assignment 3

Delivery: rush-1-3/*

Here is the expected display when calling `rush(5, 3)`:

```
Terminal
~$> clang *.c; ./a.out
ABBBA
B   B
CBBBC
```

When calling `rush(5, 1)`:

```
Terminal
~$> clang *.c; ./a.out
BBBBB
```

When calling `rush(1, 1)`:

```
Terminal
~$> clang *.c; ./a.out
B
```

When calling `rush(1, 5)`:

```
Terminal
~$> clang *.c; ./a.out
B
B
B
B
B
```


{EPITECH}

When calling `rush(4, 4)`:

Terminal

```
~$> clang *.c; ./a.out
ABBA
B  B
B  B
CBBC
```



A test binary is available next to this subject on My. **Usage: ./rush1-3 x y**

{EPITECH}

Assignment 4

Delivery: rush-1-4/*

Here is the expected display when calling `rush(5, 3)`:

```
Terminal
~$> clang *.c; ./a.out
ABBBC
B   B
ABBBC
```

When calling `rush(5, 1)`:

```
Terminal
~$> clang *.c; ./a.out
BBBBB
```

When calling `rush(1, 1)`:

```
Terminal
~$> clang *.c; ./a.out
B
```

When calling `rush(1, 5)`:

```
Terminal
~$> clang *.c; ./a.out
B
B
B
B
B
```


{EPITECH}

When calling `rush(4, 4)`:

Terminal

```
~$> clang *.c; ./a.out
ABBC
B  B
B  B
ABBC
```



A test binary is available next to this subject on My. **Usage: ./rush1-4 x y**

Assignment 5

Delivery: rush-1-5/*

Here is the expected display when calling `rush(5, 3)`:

```
Terminal
~$> clang *.c; ./a.out
ABBBC
B   B
CBBBA
```

When calling `rush(5, 1)`:

```
Terminal
~$> clang *.c; ./a.out
BBBBB
```

When calling `rush(1, 1)`:

```
Terminal
~$> clang *.c; ./a.out
B
```

When calling `rush(1, 5)`:

```
Terminal
~$> clang *.c; ./a.out
B
B
B
B
B
```


{EPITECH}

When calling `rush(4, 4)`:

```
Terminal
~$> clang *.c; ./a.out
ABBC
B  B
B  B
CBBA
```



A test binary is available next to this subject on My. **Usage: ./rush1-5 x y**

More ?

Oh... You're done?
It can be a good exercise to try and make a generic `rush_generic` function that can create any type of square on demand.
Then your `rush` functions would contain only one line which would call your `rush_generic` function with some parameters to generate the correct square.

That would remove duplicates of codes (which are generally a bad practice).

v 3.2

